(a) System / Protocol Architecture (runtime components)

PQC KEM Layer - liboqs-python (OQSKEM)

ML-KEM-512 / ML-KEM-768 / ML-KEM-1024 / FrodoKEM-976-AES | KeyGen, Encaps, Decaps
Long-term static KEM key pairs (pk_A, pk_B) are pre-provisioned; Bob generates a fresh ephemeral pair per session

uses KEM — uses KEM

ALICE (Initiator) AliceSession

Msg 1:
Encaps(pk_B) -> ct1, k1
Msg 3:
verify tau1 (key k1)
Decaps(ct2) -> k2
Encaps(pk_e) -> ct*, k*
request QKD key via mitigation layer
tau2 = MAC_k2(key_ID, ct*, ...)
Final:
verify tau3, derive key

4-Message Signature-Free HAKE

M1: ct1, nonce_A →
← M2: pk_e, tau1, ct2, nonce_B
M3: ct*, qkd_key_ID (empty if PQC-only), tau2 →
← M4: tau3

BOB (Responder) BobSession

Msg 2:
ephemeral KeyGen -> pk_e
Decaps(ct1) -> k1
Encaps(pk_A) -> ct2, k2
tau1 = MAC_k1(pk_e, nonce_B, ...)
Msg 4:
verify tau2 (key k2)
Decaps(ct*) -> k*
key_ID present -> retrieve QKD key
key_ID empty -> PQC_ONLY
tau3 = MAC_k1h(transcript ...)

Key Schedule (computed by both peers)

Transcript = (version, IDs, algorithm, nonces, pk_A, pk_B, pk_e, ct1, ct2, ct*, qkd_key_ID, security_mode) -> SHA3-256
ROKDF = SHA3-512(k*, c_KEM, k_QKD or empty, c_QKD) -> k1h || k2h; session key = k2h
tau1, tau2, tau3 = HMAC-SHA-256

QKD key request

Mitigation Layer (MitigationManager)

1. Per-peer rate quota (1 s sliding window)
2. Admission control (pool occupancy >= 5%)
3. Fallback policy: explicit PQC_ONLY or reject

mode + reason →

Handshake Result

session_key, transcript
security_mode: HYBRID_QKD | PQC_ONLY
fallback_reason: NONE | QUOTA_EXCEEDED | ADMISSION_CONTROL | QKD_UNAVAILABLE | PEER_DECLARED
Rejected handshake -> HandshakeError (no key)

GetKeyWithIDs(key_ID)

status, then GetKey (only if admitted)

Mock QKD Key Management Entity (KME) - ETSI GS QKD 014 style

(accessed by the HAKE code in-process through an injected client function)

QKDKeyPool (in-memory, thread-safe): finite pool depth, lazy refill at refill_bps, 256-bit keys, UUID key_IDs, single-use retrieval by key_ID, per-peer outstanding-key quota, errors: PoolExhausted / PeerQuotaExceeded

REST API (FastAPI): GET status, GET/POST enc_keys, GET/POST dec_keys; SAE identity via X-SAE-ID header

QKDClient (HTTP client for the REST API) implemented, not yet used by the HAKE code

(b) Experimental / Evaluation Architecture

Configuration	Experiments	Raw Results (CSV)	Plots
CLI (qkd-hake) demo benchmark --runs --warmup-runs sweep --measurement-seconds --repeats --pool-depths	**E1 Smoke Demo** pool issue/retrieve check + combiner timing (20 runs)	smoke.csv	
configs/experiments.json (rates, supply, KEMs, policy names - not read by the code)	**E2 Performance Benchmark (full HAKE)** 4 KEMs x 1000 paired runs (+100 warm-up) HYBRID vs PURE_PQC (PQC-only = explicit fallback path) mock QKD client, pool never depleted latency, CPU time, bytes on wire -> median / p95 [evaluates: HAKE + KEM layer + key schedule]	performance_benchmarks.csv	**plot_performance_figures.py** latency median/p95, message size (run separately)
	E3 WP6 QKD Starvation Sweep QKDKeyPool + simulated clock (HAKE not run) supply 1/10/100 kbps x 1-100 req/s x depth 16/64/128 Phase 1: burst depletion, measured vs theoretical Phase 2: 60 s steady-state QKD success rate (5 repeats) [evaluates: KME key pool supply vs demand]	bottleneck_steady_state.csv bottleneck_sweep.csv (legacy 10 s sweep)	**plot_wp6.py** burst vs pool depth, steady-state success, measured vs theory (run separately)

implemented & used
exists, not wired / legacy

Automated Verification (pytest)

pool & KME API, KEM round-trip, combiner, HAKE flow + tampering, WP7 mitigation regression tests (quota, admission control, explicit PQC-only fallback, fallback reasons). WP7 policy-comparison experiment: not yet implemented.